AI-ASSISTANT - 01

2303A52488

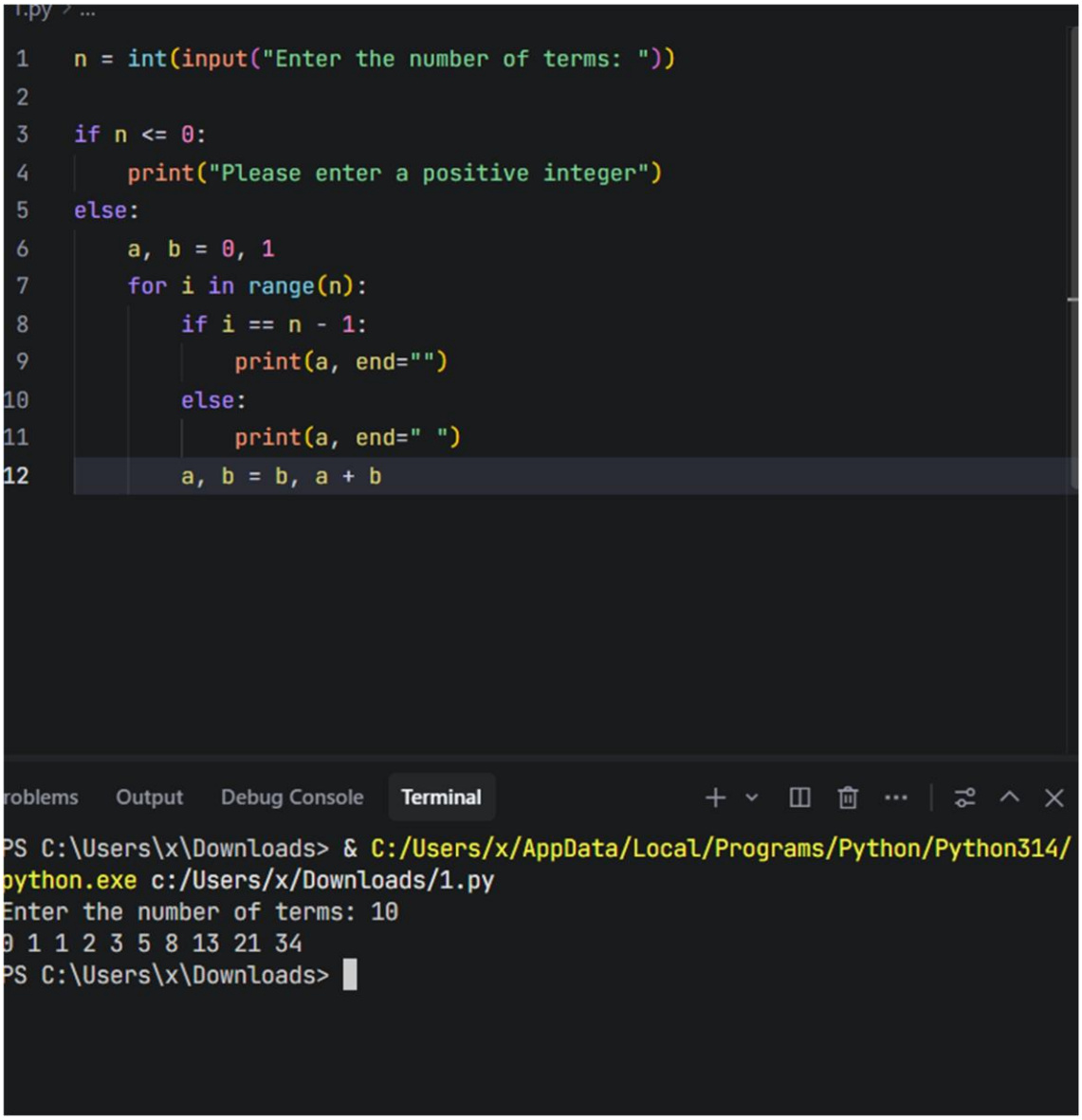
TASK 2: AI CODE OPTIMIZATION & CLEANUP (IMPROVING EFFICIENCY)

- **SCENARIO**

  THE PROTOTYPE WILL BE SHARED WITH OTHER DEVELOPERS AND NEEDS OPTIMIZATION.

- **TASK DESCRIPTION**
  - EXAMINE THE COPILOT-GENERATED CODE FROM TASK 1 AND IMPROVE IT BY:
  - REMOVING REDUNDANT VARIABLES
  - SIMPLIFYING LOOP LOGIC AVOIDING UNNECESSARY COMPUTATIONS USE COPILOT PROMPTS SUCH AS:
    - *"OPTIMIZE THIS FIBONACCI CODE"*
    - *"SIMPLIFY VARIABLE USAGE"*

IMPROVED CODE



EXPLANATION

THIS VERSION KEEPS EVERYTHING IN THE MAIN CODE (NO FUNCTIONS) BUT:

- REMOVES THE EXTRA COUNT VARIABLE
- USES A FOR LOOP DIRECTLY OVER THE NUMBER OF TERMS
- AVOIDS A SEPARATE N == 1 BRANCH
- USES A SINGLE PRINT LOCATION FOR ALL TERMS

WHAT WAS INEFFICIENT / LESS CLEAN IN THE ORIGINAL

- REDUNDANT LOOP COUNTER

- COUNT WAS TRACKING ITERATIONS, BUT PYTHON ALREADY GIVES YOU LOOP COUNTERS WITH RANGE .
- MANAGING COUNT MANUALLY (COUNT = 0 , COUNT += 1) ADDS NOISE.
- UNNECESSARY BRANCHING FOR N == 1

- SPECIAL-CASE BRANCH ELIF N == 1: PRINTED A ALONE.

- Then for larger n, the code printed a once before the loop and b inside the loop.
- Logic is split across multiple places, which reduces readability.
- Two separate print patterns

- First term (a) is printed before the loop.
- Remaining terms (b) are printed inside the loop.
- This makes it harder to reason about "what exactly is printed each iteration".
- Less direct mapping to "print n Fibonacci numbers"

- The condition while count < n - 1 is slightly less intuitive than "repeat n times".

Task 3: Modular Design Using AI Assistance (Fibonacci Using Functions)

❖ **Scenario**

The Fibonacci logic is now required in multiple modules of an application.

❖ **Task Description**

Use GitHub Copilot to generate a function-based Python program that:

- Uses a user-defined function to generate Fibonacci numbers
- Returns or prints the sequence up to *N*
- Includes meaningful comments (AI-assisted)

❖ **Expected Output**

- Correct function-based Fibonacci implementation
- Screenshots documenting Copilot's function generation

Sample Test Cases with Outputs

```python
def fibonacci(n):
    """
    Return a list containing the first n Fibonacci numbers.

    n: number of terms to generate (non-negative integer)
    """
    # Handle non-positive n by returning an empty sequence
    if n <= 0:
        return []

    # Start the sequence with the first Fibonacci number
    sequence = [0]

    # If only one term is requested, return [0]
    if n == 1:
        return sequence

    # Add the second Fibonacci number
    sequence.append(1)

    # Generate remaining terms from index 2 to n-1
    for _ in range(2, n):
        # Next term is sum of the last two terms
        next_value = sequence[-1] + sequence[-2]
        sequence.append(next_value)

    return sequence


def main():
    """Read n from the user and print the Fibonacci sequence up to n terms."""
    try:
        n = int(input("Enter the number of terms: "))
    except ValueError:
        print("Invalid input. Please enter an integer.")
        return

    fib_seq = fibonacci(n)

    if not fib_seq:
        print("No terms to display (n should be > 0).")
    else:
        print("Fibonacci sequence:")
        # Join all numbers with spaces for nice output
        print(" ".join(str(x) for x in fib_seq))


if __name__ == "__main__":
    main()
```

Test: n = 0

- Call:

```python
fibonacci(0)
```

- Output (return value):

```python
[]
```

- Running the script and entering `0`:

```
Enter the number of terms: 0
No terms to display (n should be > 0).
```

Test: n = 1

TASK 4: COMPARATIVE ANALYSIS – PROCEDURAL VS MODULAR FIBONACCI CODE

- **SCENARIO**

  YOU ARE PARTICIPATING IN A CODE REVIEW SESSION.

- **TASK DESCRIPTION**

  COMPARE THE COPILOT-GENERATED FIBONACCI PROGRAMS:

  - WITHOUT FUNCTIONS (TASK 1) □ WITH FUNCTIONS (TASK 3) □ ANALYZE THEM IN TERMS OF:
    - CODE CLARITY
    - REUSABILITY
    - DEBUGGING EASE
    - SUITABILITY FOR LARGER SYSTEMS

COMPARISON OVERVIEW

- PROCEDURAL VERSION (TASK 1) : ALL LOGIC IN MAIN , NO FUNCTIONS
- MODULAR VERSION (TASK 3) : FIBONACCI(N) FUNCTION + MAIN() WRAPPER

Aspect	Procedural (no functions)	Modular (with functions)
Code clarity	Short and linear, but mixes input, logic, and printing in one block	Separates "compute Fibonacci" from "interact with user"; intention of each part is clearer
Reusability	Hard to reuse: logic is tied to `input()` and `print()` in the same place	Easy to reuse: `fibonacci(n)` can be imported and called from any module, including tests and other components
Debugging ease	Must rerun whole script or add ad-hoc prints inside main flow	Can test `fibonacci(n)` in isolation (REPL, unit tests); easier to pinpoint bugs in the pure computation
Suitability for larger systems	Poor: scales badly as more features are added; main becomes a "god block"	Good: Fibonacci logic lives in a small, focused function that fits into layered or modular architectures
Testability	Awkward to unit-test; you test via I/O behavior or refactor first	Straightforward: write tests against `fibonacci(n)` return values without dealing with console I/O
Extensibility	Adding options (e.g., different starting values, returning vs printing) complicates main	You can extend by adding parameters or new functions while keeping existing callers mostly unchanged
Abstraction level	Low: works at the "how" level (loop and updates) only	Higher: exposes "what" (get sequence of n terms) as an interface; hides details of how it's computed
Readability for new developers	Understandable for small scripts, but logic is packed together	More readable in a team: clear entry points, docstring, and separation of concerns

SHORT ANALYTICAL REPORT

- CODE CLARITY

- THE PROCEDURAL VERSION IS FINE FOR VERY SMALL, ONE-OFF SCRIPTS: EVERYTHING IS VISIBLE IN ONE PLACE. - HOWEVER, IT MIXES THREE CONCERNS: READING INPUT, COMPUTING FIBONACCI, AND PRINTING RESULTS. AS SOON AS YOU ADD MORE FEATURES (VALIDATION, LOGGING, DIFFERENT OUTPUT FORMATS), THIS BLOCK GETS HARDER TO READ.
- THE MODULAR VERSION CLEARLY SEPARATES CONCERNS:
- FIBONACCI(N) IS ABOUT THE SEQUENCE CALCULATION.
- MAIN() IS ABOUT USER INTERACTION AND DISPLAY.
- THIS MAKES IT EASIER FOR A REVIEWER TO ANSWER "WHERE IS THE FIBONACCI LOGIC?" AND "WHERE IS THE I/O?".
- REUSABILITY

- IN THE PROCEDURAL VERSION, THE ONLY WAY TO "REUSE" THE LOGIC IS TO COPY AND PASTE THE LOOP, OR TO CALL THE SCRIPT AS A SEPARATE PROCESS. BOTH ARE FRAGILE.
- THE MODULAR VERSION PROVIDES A REUSABLE UNIT: FIBONACCI(N) CAN BE IMPORTED BY OTHER MODULES (FROM FIB IMPORT FIBONACCI) AND USED IN GUI APPS, APIS, OTHER SCRIPTS, OR TESTS.
- DEBUGGING EASE

- DEBUGGING THE PROCEDURAL CODE OFTEN MEANS SPRINKLING EXTRA PRINTS INTO THE MAIN BLOCK OR STEPPING THROUGH THE ENTIRE INPUT/PRINT FLOW IN A DEBUGGER.
- WITH THE FUNCTION-BASED DESIGN, YOU CAN:
- CALL FIBONACCI(5) DIRECTLY IN A REPL AND INSPECT THE LIST.
- ADD UNIT TESTS FOR EDGE CASES (N = 0 , N = 1 , LARGE N) WITHOUT TOUCHING I/O.
- WHEN A BUG APPEARS, YOU CAN QUICKLY DETERMINE WHETHER IT LIVES IN THE PURE LOGIC (FIBONACCI) OR IN THE USER INTERACTION (MAIN). - SUITABILITY FOR LARGER SYSTEMS

- LARGER APPLICATIONS BENEFIT FROM MODULARITY: CLEAR APIS, SMALL FUNCTIONS, AND LAYERS (UI, BUSINESS LOGIC, DATA).
- THE PROCEDURAL VERSION DOESN'T SCALE WELL BECAUSE AS MORE REQUIREMENTS ARE ADDED, THE MAIN BLOCK GROWS AND BECOMES TIGHTLY COUPLED TO EVERYTHING ELSE.
- THE MODULAR VERSION MAPS NATURALLY INTO A LARGER ARCHITECTURE:
- FIBONACCI(N) IS PART OF A "DOMAIN" OR "CORE" MODULE.
- DIFFERENT FRONT-ENDS (CLI, WEB, GUI) CAN ALL REUSE THE SAME CORE FUNCTION.

TASK 5: AI-GENERATED ITERATIVE VS RECURSIVE FIBONACCI APPROACHES (DIFFERENT ALGORITHMIC APPROACHES FOR FIBONACCI SERIES)

❖ **SCENARIO**

YOUR MENTOR WANTS TO ASSESS AI'S UNDERSTANDING OF DIFFERENT ALGORITHMIC PARADIGMS.

❖ **TASK DESCRIPTION**

PROMPT GITHUB COPILOT TO GENERATE:

AN ITERATIVE FIBONACCI IMPLEMENTATION

A RECURSIVE FIBONACCI IMPLEMENTATION

```
ITERATIVE DEF
FIB_ITERATIVE(
N):    IF N < 0:
      RAISE VALUEERROR("N MUST BE NON-NEGATIVE")

  A, B = 0, 1
FOR _ IN
RANGE(N):
A, B = B, A + B
  RETURN A

RECURSIVE
DEF
FIB_RECURSIVE(N)
:    IF N < 0:
      RAISE VALUEERROR("N MUST BE NON-NEGATIVE")

  IF N == 0:
RETURN 0
IF N == 1:
RETURN 1

  RETURN FIB_RECURSIVE(N - 1) + FIB_RECURSIVE(N - 2)
```

EXECUTION FLOW

- ITERATIVE

- INITIALIZE A = 0 , B = 1 .
- LOOP N TIMES:
- SET (A, B) TO THE NEXT FIBONACCI PAIR (B, A + B) .
- AFTER THE LOOP, A HOLDS FIB(N) .
- CONTROL FLOW IS STRAIGHTFORWARD: ONE LOOP, NO CALL STACK GROWTH.
- RECURSIVE

- FOR EACH CALL FIB_RECURSIVE(N) :

- If n is 0 or 1, return the base value directly.
- Otherwise, call itself twice: - fib_recursive(n - 1)
- fib_recursive(n - 2)
- Sum the two results.
- The call tree branches until all paths hit n == 0 or n == 1 .

Complexity and Performance Comparison

- Time Complexity

- Iterative:
- Each loop iteration does O(1) work.
- Loop runs n times.
- Time complexity: O(n) .
- Naive recursive:
- Each call branches into two more calls (except at base cases).
- Number of calls grows roughly like 2^n .
- Time complexity: O(2^n) (exponential).
- Space Complexity

- Iterative:
- Uses a constant number of variables (a , b , loop index).
- Space complexity: O(1) auxiliary space.
- Naive recursive:
- Each call waits on the result of two child calls.
- Maximum recursion depth is n (for the longest path down to 0/1).
- Space complexity: O(n) due to call stack.
- Performance for Large n

- Iterative:
- Scales well up to very large n (e.g., hundreds of thousands) until integers or time become the bottleneck.
- Work grows linearly; practical for most uses.
- Naive recursive:
- Becomes unusable quickly:
- For n ~ 30–40 , it is already noticeably slow.
- For n > 40–45 , runtime explodes because of exponential growth in calls.

When Recursion Should Be Avoided

- You need to handle large n .
- You are in a performance-sensitive path (e.g., called many times in a loop or server endpoint). - The recursive formulation introduces overlapping subproblems (like Fibonacci), causing repeated work and exponential behavior.
- Your runtime has a strict recursion depth limit that the algorithm might hit.